

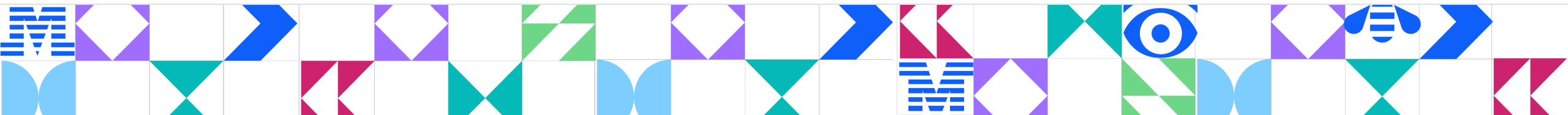


AI-Ready Context

watsonx 위에 올린 설명가능한
Agentic RAG + Knowledge Graph

질문마다 벡터 · 그래프 · **SQL** 도구로 자동 라우팅하고, 답의 근거를 그대로 보여주는 데모.

Kiyeon Jeon · AI Engineer, IBM Client Engineering Korea
IBM TechXchange Korea 워크숍



목차

- 배경 — 왜 Agentic RAG + Knowledge Graph인가
- 아키텍처 & 데이터 — 스토어 구성 · 동작 방식 · 문서 적재 · 데모 코퍼스
- 데모 — 3경로 라우팅 · No-code 즉석 적재 · 설명가능성
- 고도화 — 각 도구를 한 단계 더 (text-to-SQL · KG · 혼합문서 · Langflow)
- 심화 — KG를 그래프DB가 아닌 NoSQL(AstraDB)에 담은 법
- 운영 — 검색·KG 품질 개선 · 배포 · 저장소 연결 · 한계
- 정리 & Q&A

왜 이걸 만들었나

- **RAG의 한계** — 벡터 검색 하나로는 못 푸는 질문이 많다
 - "총 몇 개 문서가 있나?" → 검색이 아니라 집계(**SQL**)
 - "A 법과 B 기관의 관계는?" → 의미 유사도가 아니라 관계(**그래프**)
 - **신뢰 문제** — LLM 답이 맞는지 어떻게 믿나? → 근거(**grounding**)를 보여줘야 한다
 - **운영 현실** — 폐쇄망·규제 산업은 SaaS LLM을 그냥 못 쓴다 → **watsonx**로 거버넌스
- "질문에 맞는 도구를 고르고, 근거를 보여주는" 에이전틱 RAG

세 가지 포인트

01

에이전트 라우팅

Langflow의 watsonx granite 에이전트가 **vector / graph / sql** 도구를 tool-calling으로 선택. 어떤 도구를 썼는지 **tool-trace** 배지로 표시.

02

설명가능성

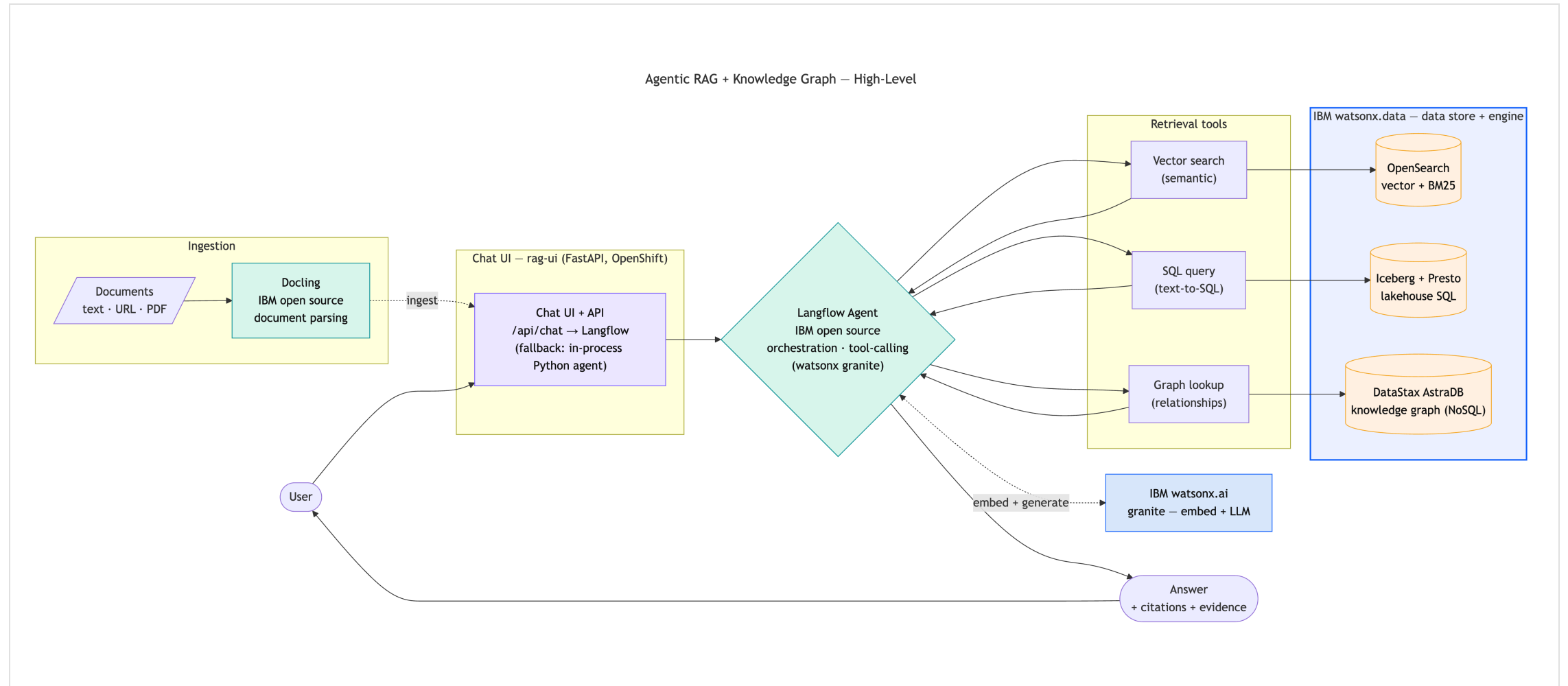
답변마다 인용 + 근거 패널. 검색 청크 · KG 서브그래프 · SQL 결과를 펼쳐서 확인.

03

No-code 즉석 적재

UI에서 문서를 텍스트/URL/PDF로 추가 → 답이 즉시 바뀌고, 삭제하면 원복. "근거 있으면 정확, 없으면 환각"을 클릭만으로.

한 질문, 세 갈래 검색 — watsonx 스택



스택 — 전부 IBM

역할	구성요소
Chat UI	rag-ui — FastAPI (OpenShift) · tool-trace·근거 패널
오케스트레이션	Langflow 에이전트 (IBM open source, tool-calling) · 파이썬 폴백
벡터 검색	OpenSearch (IBM watsonx.data) — kNN + BM25 하이브리드
지식그래프	DataStax AstraDB (IBM watsonx.data) — 엔티티(+emb)/엣지
text-to-SQL	IBM watsonx.data Iceberg + Presto — AML 데이터셋
임베딩 + LLM	IBM watsonx.ai granite
문서 파싱	Docling (IBM open source)

한 문서, 세 가지 형태

스토어	무엇을 담나	어떤 질문에 강한가
OpenSearch	청크 + 임베딩(768d) + 원문 텍스트	"가명정보란 무엇인가" (의미 검색)
AstraDB (KG)	엔티티(임베딩) · 관계(엣지)	"접근매체와 감독기관의 관계" (관계)
Iceberg / Presto	문서서 추출한 의무·조항·벌칙 표 (+ 별도 AML 데이터셋)	"법별 의무 건수" · "위험등급 high 거래 총액" (text-to-SQL)

- 핵심: 같은 문서를 벡터·그래프·SQL 세 형태로 저장 → 질문에 맞춰 골라 읽는다
- 세 스토어 모두 **IBM watsonx.data** 패밀리 · OpenSearch는 벡터+BM25를 **RRF**로 융합
- SQL은 문서에서 추출한 의무 테이블 + 별도 **AML** 비즈니스 데이터셋 둘 다 질의

질문이 들어오면

질문

- [라우터] granite가 도구 선택 {vector, graph, sql} (+관계형 키워드 백스톱)
- vector : OpenSearch 하이브리드 (kNN + BM25, RRF 융합, k=8)
 - graph : AstraDB KG - 엔티티 임베딩 시드 → 1홉 이웃 조회 → 별칭 병합
 - sql : text-to-SQL - granite가 Presto SELECT 생성 → 가드 → 실행 (AML)
- [합성] granite가 컨텍스트로 답변 생성 + 인용
- [UI] 답변 + tool-trace 배지 + 근거 패널 (생성 SQL 포함)

- 챗 UI(/api/chat)는 오케스트레이션을 **Langflow** 에이전트에 위임 — 오류 시 파이썬 에이전트로 폴백
- 라우팅·검색·합성·임베딩은 전부 **watsonx.ai granite** · 답변 언어는 질문 언어를 따름

한 번에 다섯 저장소

```
ingest_source(문서)  doc_id = sha1(source)
├─ OpenSearch        : 청크 + granite 임베딩(768d)      ← vector
├─ AstraDB kg        : granite 추출 엔티티(+emb)/엣지    ← graph
├─ Iceberg obligations : granite 추출 의무·조항·벌칙 표  ← sql
├─ AstraDB doc_registry: 제목·소스·해시·카운트          ← 추적/증분
└─ Iceberg corpus     : 문서 인벤토리 1행              ← 추적
```

- 멍등 **upsert**(doc_id 기준) + **content-hash** 스킵 → 재적재해도 중복 없음
- **CronJob rag-reindex**(30분): 등록 소스 재적재, 바뀐 것만 · 파싱은 **docling-serve**(텍스트/URL/PDF)

한글 금융·컴플라이언스 — md · PDF · URL 혼합

- **md (4)** — 개인정보 보호법 · 마이데이터 · 전자금융거래법 · 전자금융감독규정
- **PDF (2)** — 신용정보법 · 특정금융정보법 (docling 변환 적재)
- **URL (2)** — Docling · Langflow README (데모가 쓰는 오픈소스 도구 문서 · docling fetch)

왜 금융 법령? 법령 간 교차참조가 많아 지식그래프가 "의미를 갖는" 도메인. md/PDF/URL 혼합으로 문서 처리 다양성과 한글/영문 다국어를 동시 시연하고, 별도 **AML** 데이터셋 (iceberg_data.aml)으로 text-to-SQL을 보여준다.

URL 문서는 이 데모가 실제로 쓰는 오픈소스(**Docling·Langflow**) **README** → "Docling 문서 포맷은?"처럼 시스템이 자기 스택을 스스로 설명하는 질문도 된다.

어떤 데이터를 넣었나

문서	형식	출처 · 적재 경로
개인정보 보호법 (PIPA)	md	corpus/01_개인정보보호법.md
마이데이터 (본인신용정보관리업)	md	corpus/03_마이데이터.md
전자금융거래법	md	corpus/04_전자금융거래법.md
전자금융감독규정	md	corpus/05_전자금융감독규정.md
신용정보법	PDF	corpus_pdf/02_신용정보법.pdf · docling 변환
특정금융정보법 (AML/CFT)	PDF	corpus_pdf/06_특정금융정보법.pdf · docling 변환
Docling README	URL	GitHub raw · docling fetch
Langflow README	URL	GitHub raw · docling fetch

+ 문서서 추출되는 정형 테이블 rag.obligations(law·party·obligation·penalty) — 적재 때 granite가 의무·벌칙을 뽑아 text-to-SQL로 질의. + 별도 **AML** 데이터셋(사전 시드·문서와 무관) iceberg_data.aml — customers·accounts·transactions·str_reports.

세 가지 검색 경로

경로	예시 질문	보여줄 것
vector	"가명정보란 무엇이고 동의 없이 쓸 수 있는 목적은?"	인용칩 + 검색 청크(유사도 바)
graph	"접근매체와 감독기관의 관계를 지식그래프로 보여줘"	엔티티 칩 + 엣지 트리플
sql	"위험등급 high 고객의 플래그 거래 총액을 상대국가별로?"	생성 SQL 코드블록 + 결과 표
hybrid	"자금세탁 의심거래는 어디 보고하고, VASP 의무는?"	vector+graph 배지 동시 점등

→ 각 답변에서 **tool-trace** 배지와 근거 패널을 펼쳐 보여준다.



관계 → 지식그래프 서브그래프

Lakehouse Assistant
Agentic RAG · Knowledge Graph · watsonx

OpenSearchAstraDBwatsonx.datawatsonx.ai

☰ 코퍼스

전자금융거래법의 접근매체와 감독기관의 관계는?

전자금융거래법은 접근매체를 통해 전자금융거래를 수행하며, 이를 감독하는 기관은 금융위원회이며, 감사는 금융감독원이 수행한다. 또한, 전자금융거래의 청산·결제에 대해서는 한국은행이 감시(oversight) 기능을 수행한다.

관련 문서: 전자금융거래법#0

전자금융거래법#0전자금융감독규정#0특정금융정보법(자금세탁방지)#0마이데이터(본인신용정보관리법)#0마이데이터(본인신용정보관리법)#1특정금융정보법(자금세탁방지)#1전자금융거래법#1전자금융감독규정#1

TOOLS

vector searchknowledge graphsql / iceberg

Retrieved passages 8

Knowledge subgraph 32

마이데이터 service접근매체 medium전자금융감독규정 regulation전자금융거래법 law전자금융거래 transaction금융회사 entity금융감독원 organization개인정보 보호법 law전자금융업자 entity이상거래탐지시스템 FDS (technology)5·5·7 guideline접근통제·계정관리 technology정보보호최고책임자 CISO (role)고객현금거래보고 CTR (procedure)금융보안원 regulatory_body금융위원회 organization

마이데이터 — related_to → 접근매체
전자금융감독규정 — based_on → 전자금융거래법
접근매체 — used_in → 전자금융거래
금융회사 — reports_to → 금융감독원

Ask the lakehouse...

전자금융거래법 — regulates → 전자금융업자

자연어 → 생성 SQL + 결과

Lakehouse Assistant
Agentic RAG · Knowledge Graph · watsonx

OpenSearch AstraDB watsonx.data watsonx.ai

인덱싱된 문서는 총 몇 개이고 각각 체크 수는?

인덱싱된 문서는 총 8개이고, 각각의 체크 수는 다음과 같습니다:

- Docling README: 9개
- Langflow README: 6개
- 마이데이터(본인신용정보관리법): 2개
- 신용정보법: 2개
- 전자금융거래법: 2개
- 개인정보 보호법: 2개
- 특정금융정보법(자금세탁방지): 2개
- 전자금융감독규정: 2개

TOOLS

- vector search
- knowledge graph
- sql / iceberg

Corpus / SQL rows 8

Docling README	https://raw.githubusercontent.com/docling-project/docling/main/README.md	9	14	14
Langflow README	https://raw.githubusercontent.com/langflow-ai/langflow/main/README.md	6	15	15
마이데이터 (본인신용정보관리법)	/corpus/03_mydata.md	2	11	9
신용정보법	/corpus/02_credit.md	2	11	12
전자금융거래법	/corpus/04_efta.md	2	11	10
개인정보 보호법	/corpus/01_pipa.md	2	12	10
특정금융정보법 (자금세탁방지)				

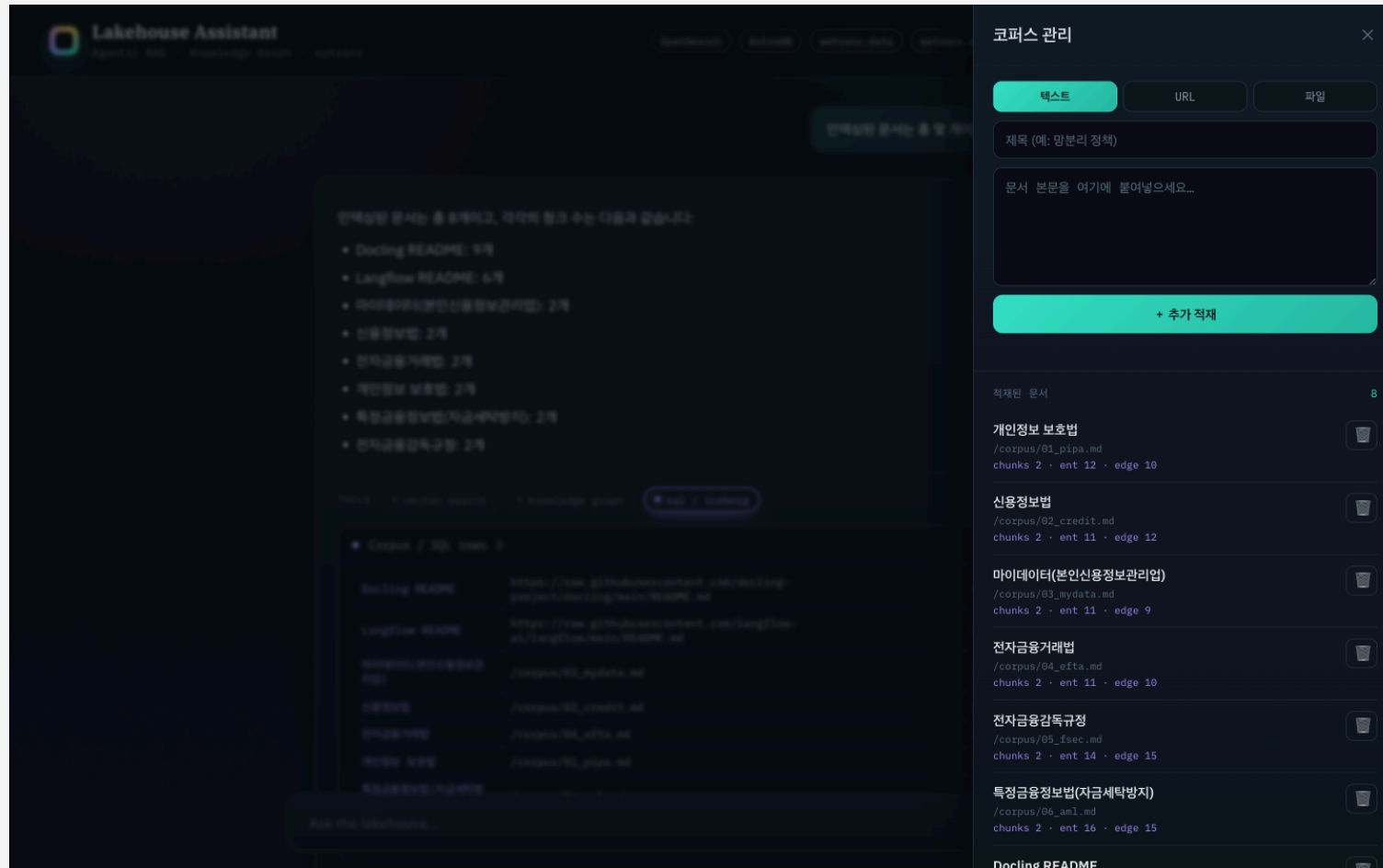
Ask the lakehouse...

No-code 즉석 적재 → 답 변화 → 원복

- 적재 전 질문 — "CSAP는 어느 기관이 평가하나?" → **환각**(코퍼스에 없어 엉뚱한 답)
- ⚙ 코퍼스 드로어 → 텍스트 탭 → 제목+본문 붙여넣기 → 추가 적재 (목록 8 → 9)
- 같은 질문 재질의 → **정확** + 새 문서 인용 (KISA·한국인터넷진흥원)
- 🗑 삭제 → 같은 질문 → 다시 **환각으로 원복**

입력 3종: 텍스트(주력) · **URL**(docling fetch) · 파일(PDF/DOCX). 한 번의 적재로 벡터·그래프·SQL(의무표)·추적까지 동시 반영.

코드 한 줄 없이 코퍼스 변경



근거를 어떻게 보여주나

- **tool-trace** 배지 — 이 답에 vector / graph / sql 중 무엇을 썼는지
- 인용 칩 — 답변 속 주장 ↔ 출처 문서 연결
- 근거 패널(펼치기) — Retrieved passages(유사도) · Knowledge subgraph(엔티티·엣지) · Generated SQL + 결과
- 출처 링크 — URL 문서는 원문, 직접작성 md는 /corpus/*.md로 열림

신뢰의 핵심은 "왜 이렇게 답했는지"를 보여주는 것. 배지로 도구를, 패널로 실제 근거를, 출처는 클릭하면 원문이 열린다 — 블랙박스가 아니다.

각 도구를 한 단계 더

도구	Before (앞음)	After (이번 고도화)
SQL	corpus 메타데이터 고정 쿼리	text-to-SQL — AML 데이터셋 + 문서서 추출한 의무 테이블에 granite가 SELECT 생성·실행 (SELECT-only 가드 + self-correction), 생성 SQL 노출
KG	전체 로드 + 문자열 매칭	엔티티 임베딩 시드 + 엔티티 해소 + 타입 온톨로지 → 1홉 서브그래프
문서	md 6종	md + PDF + URL 혼합 (docling 변환)
오케스트레이션	파이썬 단일패스 라우터	Langflow 에이전트가 메인 (tool-calling, 파이썬은 폴백)

공통: 라우터에 관계형 키워드 백스톱 · 원칙: 데모는 명료·안정 / 프로덕션 경로는 정직하게(README 대조표).

실제 비즈니스 데이터에 자연어 질의

- 데이터셋 2종: aml.*(고객·거래·STR, 사전 시드) + rag.obligations(문서서 추출한 의무·조항·벌칙) — granite가 질문에 맞는 쪽을 선택

질문 → granite가 Presto SELECT 생성 (스키마카드 + few-shot)
→ 가드 (SELECT-only · 자동 LIMIT · DDL 차단)
→ 실행, 에러 시 self-correction 1회
→ 결과 + 생성 SQL 을 패널에 노출

예: "위험등급 high 고객의 플래그 거래 총액을 상대국가별로?" → 3-테이블 조인 SQL 자동 생성·실행. 프로덕션: 시맨틱 레이어/dbt · 행수준 보안 · 쿼리 검증.

오케스트레이션을 눈으로

```
ChatInput → Agent (IBM watsonx.ai) → ChatOutput
          ▲ tools
          search_documents(vector) · lookup_relationships(graph) · query_business_data(SQL)
```

- rag-ui가 도구를 **HTTP** 엔드포인트로 노출 (/api/tool/{vector,graph,sql}, 토큰 게이트)
- 챗 UI → Langflow 실행 → 응답의 tool 출력으로 근거 패널 그대로 복원
- 대화별 **session_id**(브라우저 uuid)로 멀티턴 + 사용자 격리 · "새 대화" 리셋 · 실패 시 파이썬 폴백

그래프DB가 아닌 NoSQL에 담기

코퍼스가 작고(8문서·~100노드/108엣지) 질의는 1홉 서브그래프뿐 → 멀티홉 엔진 불필요. 단일 컬렉션 kg에 kind로 노드/엣지 구분.

```
{ "kind": "entity", "name": "마이데이터", "type": "service_provider",  
  "norm": "마이데이터", "emb": [...768d...], "doc_id": "..." }  
  
{ "kind": "edge", "src": "마이데이터", "rel": "based_on", "dst": "신용정보법",  
  "src_norm": "마이데이터", "dst_norm": "신용정보법" } // 정규화 트리플
```

emb = 768d 임베딩(시드·해소) · *_norm = 정규화 키(별칭 병합) · rel 통제어휘 14종 · type 온톨로지 11종.

벡터 시드 + 1홉 이웃 조회

```
# 적재(write) - 문서 1건 → 같은 doc_id로 삭제 후 재삽입
granite {entities(type), edges} 추출 → 임베딩(emb) + 엔티티 해소(norm + 코사인≥0.90)
→ astra_delete_all(doc_id) → insertMany    # 먹등

# 질의(read) - 벡터 시드 + 1홉
질문 임베딩 ↔ 엔티티 emb 코사인 → 의미적 시드 선정
→ 1홉 엣지  find({kind:edge, $or:[src_normES, dst_normES]})
→ 점수=질문언급(+2)+시드(+1) → 별칭 병합·자기루프 제거 → 칩 + 트리플
```

△ 이 AstraDB는 서버사이드 벡터 ANN 미지원(SAI 'aa') → KG가 작아 코사인 앱-사이드. 스케일·트래버설은 CQL 파티션 / 그래프DB.

PoC를 쓸 만하게 만든 디테일

- 하이브리드 검색 — 벡터(의미) + BM25(정확매칭) RRF 융합 → 약어·법령명(STR·VASP·CISO)에 강함
- **cjk** 분석기 — 한글 bigram 토큰화 → 한글 BM25 recall 개선 (nori 미설치 우회)
- **KG** 정규화·해소 — 정규화 + 임베딩 코사인 별칭 병합 · 타입 온톨로지 · 관계 snake_case
- **text-to-SQL** 가드 — read-only(SELECT-only + LIMIT) · 실행 에러 self-correction 1회
- 라우터 백스톱 — 관계형 키워드로 graph 경로 보강 (granite 과소선택 보완)

이미지 빌드 없이

- 내부 레지스트리 **Removed** 환경 → 이미지 빌드 불가
- 해법: **ConfigMap + 런타임 pip-install** — 코드/정적파일 마운트, ubi9/python-312가 기동 시 설치·실행
- 단일 컨테이너 FastAPI · 자격증명은 rag-secrets · 그래프 질의 30s 초과 → 라우트 타임아웃 120s

```
oc -n genai-apps create configmap rag-code --from-file=*.py --dry-run=client -o yaml | oc apply -f -
oc -n genai-apps rollout restart deploy/rag-ui # 이미지 빌드 없음
```

vector와 graph는 어떻게 연결되나

질문

```
├ vector → OpenSearch 청크 검색
└ graph → AstraDB 서브그래프 검색  ┘ → 두 결과를 이어붙여 LLM 컨텍스트 → 합성
공통 키 = doc_id (문서 단위) · 청크↔엔티티 직접 링크는 없음
```

지금 — 느슨한 결합

- 두 검색이 서로 참조하지 않고 각자 수행(평행) → 융합은 **LLM 프롬프트** 단계에서
- 공통 키는 doc_id(문서 단위)뿐 — 엔티티↔청크 링크 없음
- 데모 목적엔 충분: LLM이 청크+서브그래프를 함께 보고 답 · 근거 패널에 병치

매끄럽게 하려면 — 운영

- 청크 단위 키 — 엔티티에 chunk_no(=text_unit_ids) → 벡터히트 → 엔티티 → **1**홉 연쇄
- **lexical graph** — (:Chunk)-[:HAS_ENTITY]->(:Entity)로 청크↔그래프 조인
- 단일 벡터+그래프 저장소 — Neo4j 노드 벡터 인덱스 / 벡터-AstraDB
\$vector

지금은 **doc_id** 기반 느슨한 결합 + **LLM** 컨텍스트 융합 · 운영은 청크 단위 링크로 **vector→graph** 연쇄가 정석.

알아둘 한계 (데모 수용 범위)

- 무인증 쓰기 — 적재/삭제 공개. 운영 시 인증·outbox 트랜잭션 필요
- 라우터 비결정성 — granite가 graph 과소선택 → 키워드 백스톱 보강(완전 결정적은 아님)
- 그래프 순회 아님 — 1홉 서브그래프 수준. AstraDB 비벡터 NoSQL → 벡터 시드는 앱-사이드 코사인
- **text-to-SQL** 범위 — read-only 가드; 복잡 조인은 few-shot로 유도(완전 일반화 X)
- **docling-graph** 미사용 — KG는 granite LLM 추출. docling-graph는 스캔PDF/복잡표용 옵션

AI에게 딱 한 글자,
"배" 라고만 물으면?

무엇을 떠올릴까요? 🤔

AI에게 딱 한 글자, "배" 라고만 물으면?



타는 배
ship



먹는 배
pear



아픈 배
stomach

자, 정답은...? 🤔

핵심 메시지

Agentic RAG

검색 하나가 아니라, 질문에 맞는 도구를 고르는 것

Explainability

답뿐 아니라 근거를 보여줘야 신뢰가 생긴다

AI-Ready Context

같은 문서를 벡터·그래프·SQL 세 형태로 준비

watsonx + OpenShift

규제 산업에서도 거버넌스 가능한 스택

데모 https://rag-ui-genai-apps.apps.<CLUSTER_DOMAIN>
코드 github.com/kiyeonjeon21/techxchange-ai-ready-context

기술 노트

- watsonx.data Presto는 **PrestoDB** (presto-python-client, trino 아님)
- OpenSearch: faiss 없음 → **lucene kNN**, 한글 BM25는 **cjk** 분석기
- KG: granite 추출 + 임베딩(emb) 엔티티 해소 + 타입 온톨로지; 1홉은 \$or/\$in 서버 필터
- text-to-SQL: 스키마카드+few-shot, SELECT-only 가드, self-correction (iceberg_data.aml)
- 모델: granite (챗 엔진) + granite-embedding-278m (768d) · Langflow는 **granite-4-h-small**
- 파싱: docling-serve /v1/convert/source(URL) · /v1/convert/file(PDF)

상세: docs/architecture.pdf · docs/DEMO.md · NOTE.md

저장소 연결 & 중복 처리

```
doc_id = sha1(source)[:16]    // 공통 키. source = URL | file:파일명 | inline:제목
OpenSearch      chunk _id = "{doc_id}-{i}"      + doc_id 필드
AstraDB kg      _id = "{doc_id}:e|r:{i}"        + doc_id 필드
AstraDB registry _id = doc_id    // 중심 인덱스: 제목·소스·해시·카운트
Iceberg corpus / obligations      doc_id 컬럼
```

중복 방지 — 2단계 (먹등)

- **content-hash** 스킵 — sha256(md)가 그대로면 임베딩·LLM 추출까지 생략(unchanged)
- **doc_id** 먹등 **upsert** — 각 저장소에서 "그 doc_id 삭제 → 재삽입" → 재적재해도 행 누적 없음
- 삭제도 doc_id 하나로 다섯 저장소 일괄(delete_doc)

알아둘 한계 (데모 범위)

- doc_id는 **source** 문자열 해시(내용 아님) → 다른 경로로 같은 문서면 중복 적재
- KG 엔티티는 **doc별** 보관 + **norm** 공유(질의 시 병합), 저장 단계 단일화 아님
- 세 저장소 교차 트랜잭션 없음(best-effort) → 운영은 **outbox**/트랜잭션 필요

정리: 연결은 **doc_id** 단일 키 · 중복은 **content-hash + doc_id** 먹등 **upsert**로 해결.

큰 문서는? — 실무 GraphRAG의 청크 단위 추출

항목	우리 데모	실무 GraphRAG (MS · LlamaIndex · Neo4j)
추출 단위	문서 1회 (md[:3500])	청크(text unit)별 LLM 호출 — 기본 ~1,200토큰, 설정 가능
출처(provenance)	doc_id (문서 단위)	엔티티·관계마다 text_unit_ids(청크 ID 목록) 기록
누락 방지	없음 (앞부분만)	gleaning — 같은 청크 다중 추출 패스
엔티티 병합	norm+코사인 (문서 간)	title+type 병합 → description 배열 LLM 요약
청크↔엔티티	링크 없음	그래프에 (:Chunk)-[:HAS_ENTITY]->(:Entity)
상위 구조	1홉 서브그래프	커뮤니티 탐지(Leiden) + 커뮤니티 요약

핵심: 큰 문서는 문서 단위가 비현실적 → 청크 단위 추출이 표준이고, 각 엔티티/관계에 출처 청크 ID를 남긴다. 우리도 엔티티/엣지에 chunk_nos를 더하면(= text_unit_ids) 3500자 한계 해소 + "근거 청크"까지 설명가능. 비용은 LLM 호출이 문서당 1회→청크 수만큼 증가(청크 묶음으로 조절).

출처: Microsoft GraphRAG (dataflow) · LlamaIndex PropertyGraphIndex · Neo4j LLM Graph Builder.

KG 벡터 검색, 노드가 늘면? — $O(n) \rightarrow ANN$

지금: emb를 AstraDB 일반 필드로 저장 → 앱에서 전수(**brute-force**) 코사인
질의 : 전체 엔티티 fetch (**Data API 20건/page = n/20 왕복**) + $O(n \times 768)$ 코사인
적재해소 : 신규 m개 × 기존 n개 전쌍 = $O(m \times n \times 768)$
→ n(노드 수)에 선형 · ~100노드 즉시 · ~1k 둔화 · ~10k+ 비현실적

왜 작은 KG 전용인가

- **ANN** 아님 = 정확 전수 계산 → n 커지면 지연 급증
- 진짜 병목은 코사인보다 매 질의 전체 **fetch**(20/page 왕복)
- 이 AstraDB('aa')는 서버사이드 벡터 인덱스 자체가 불가

운영 전환

- 시드 검색 → **ANN** 인덱스: OpenSearch kNN(이미 보유)·pgvector·벡터-Astra \$vector → sublinear·top-k만
- 엔티티 해소 → 블로킹/**LSH** 또는 **ANN** 후보생성(전쌍 회피)
- 1홉/멀티홉 → **CQL** 파티션 키 / 그래프DB(Neo4j)
- 대규모 → 커뮤니티 사전요약(GraphRAG식)

이 스택 1순위: 엔티티 **emb**를 **OpenSearch kNN**으로 이전 → 전수 계산·전체 fetch 제거(코드 변경 작음). 규모: <1k 현행 OK · 1k~100k ANN+블로킹 · 100k+ 벡터DB+그래프DB+오프라인 ER.